

Correzioni codice MediaPipe



```
import cv2
import mediapipe as mp
import time
import math as math

class HandTrackingDynamic:
    def __init__(self, mode=False, maxHands=2, detectionCon=0.5, trackCon=0.5):
        self.__mode__ = mode
        self.__maxHands__ = maxHands
        self.__detectionCon__ = detectionCon
        self.__trackCon__ = trackCon
        self.handsMp = mp.solutions.hands
        self.hands = self.handsMp.Hands()
        self.mpDraw= mp.solutions.drawing_utils
        self.tipIds = [4, 8, 12, 16, 20]

    def findFingers(self, frame, draw=True):
        imgRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        self.results = self.hands.process(imgRGB)
        if self.results.multi_hand_landmarks:
            for handLms in self.results.multi_hand_landmarks:
                if draw:
                    self.mpDraw.draw_landmarks(frame, handLms,self.handsMp.HAND_CONNECTIONS)

        return frame

    def findPosition( self, frame, handNo=0, draw=True):
        xList=[]
        yList=[]
        bbox = []
        self.lmsList=[]
        if self.results.multi_hand_landmarks:
            myHand = self.results.multi_hand_landmarks[handNo]
            for id, lm in enumerate(myHand.landmark):

                h, w, c = frame.shape
                cx, cy = int(lm.x * w), int(lm.y * h)
                xList.append(cx)
                yList.append(cy)
                self.lmsList.append([id, cx, cy])
                if draw:
                    cv2.circle(frame, (cx, cy), 5, (255, 0, 255), cv2.FILLED)
```

```

    xmin, xmax = min(xList), max(xList)
    ymin, ymax = min(yList), max(yList)
    bbox = xmin, ymin, xmax, ymax
    print( "Hands Keypoint")
    print(bbox)
    if draw:
        cv2.rectangle(frame, (xmin - 20, ymin - 20),(xmax + 20, ymax + 20),
            (0, 255 , 0) , 2)

    return self.lmsList, bbox

def findFingerUp(self):
    fingers=[]

    if self.lmsList[self.tipIds[0]][1] > self.lmsList[self.tipIds[0]-1][1]:
        fingers.append(1)
    else:
        fingers.append(0)

    for id in range(1, 5):
        if self.lmsList[self.tipIds[id]][2] < self.lmsList[self.tipIds[id]-2][2]:
            fingers.append(1)
        else:
            fingers.append(0)

    return fingers

def findDistance(self, p1, p2, frame, draw= True, r=15, t=3):

    x1 , y1 = self.lmsList[p1][1:]
    x2, y2 = self.lmsList[p2][1:]
    cx , cy = (x1+x2)//2 , (y1 + y2)//2

    if draw:
        cv2.line(frame,(x1, y1),(x2,y2) ,(255,0,255), t)
        cv2.circle(frame,(x1,y1),r,(255,0,255),cv2.FILLED)
        cv2.circle(frame,(x2,y2),r, (255,0,0),cv2.FILLED)
        cv2.circle(frame,(cx,cy), r,(0,0.255),cv2.FILLED)
    len= math.hypot(x2-x1,y2-y1)

    return len, frame , [x1, y1, x2, y2, cx, cy]

def main():

    ctime=0
    ptime=0
    cap = cv2.VideoCapture(0)
    detector = HandTrackingDynamic()
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    if not cap.isOpened():

```

```

print("Cannot open camera")
exit()

while True:
    ret, frame = cap.read()

    frame = detector.findFingers(frame)
    lmsList, _ = detector.findPosition(frame)
    if len(lmsList) != 0:
        # Uncomment the following line if you want to print the first landmark
        # print(lmsList[0])

        ctime = time.time()
        fps = 1 / (ctime - ptime)
        ptime = ctime

    cv2.putText(frame, str(int(fps)), (10,70), cv2.FONT_HERSHEY_PLAIN,3,(255,0,255),3)

    #gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    cv2.imshow('frame', frame)
    cv2.waitKey(1)

if __name__ == "__main__":
    main()

```

Il tuo codice per il rilevamento delle mani con MediaPipe sembra ben strutturato, ma ci sono alcuni miglioramenti e correzioni che potrebbero essere utili:

1. Correggere l'errore di `ptime`

- Nel `main()`, la variabile `ptime` viene inizializzata a 0 prima del loop, il che causa un errore alla prima iterazione (`fps = 1 / (ctime - ptime)` sarà enorme). Dovresti inizializzare `ptime = time.time()` prima del loop.

2. Errore nella conversione RGB/BGR

- MediaPipe lavora con immagini in formato RGB, ma OpenCV usa BGR. Nel metodo `findFingers()`, stai convertendo in RGB prima di elaborare, ma poi disegni sull'immagine BGR originale. Questo va bene, ma potresti rendere più chiaro il flusso dei dati.

3. Uso di `self.results` in `findPosition()`

- Se `findPosition()` viene chiamato dopo `findFingers()`, tutto ok. Ma se viene chiamato senza aver aggiornato `self.results`, potrebbe generare errori. Sarebbe utile un controllo `if self.results is None:` prima di usarlo.

4. Errore nell'uso di `math.hypot()`

- Nel metodo `findDistance()`, la variabile `len` viene definita, ma è una parola chiave in Python. Meglio rinominarla, ad esempio `distance`.

5. Errore di indicizzazione in `findFingerUp()`

- Il codice assume sempre che `self.lmsList` abbia tutti gli elementi, ma se la mano non è rilevata, si verificherà un `IndexError`. Prima di accedere agli indici, verifica che `self.lmsList` non sia vuota.

Versione migliorata del codice con fix

```
python

`import cv2
import mediapipe as mp
import time
import math

class HandTrackingDynamic:
    def __init__(self, mode=False, maxHands=2, detectionCon=0.5, trackCon=0.5):
        self.mode = mode
        self.maxHands = maxHands
        self.detectionCon = detectionCon
        self.trackCon = trackCon

        self.handsMp = mp.solutions.hands
        self.hands = self.handsMp.Hands(static_image_mode=self.mode,
                                         max_num_hands=self.maxHands,
                                         min_detection_confidence=self.detectionCon,
                                         min_tracking_confidence=self.trackCon)

        self.mpDraw = mp.solutions.drawing_utils
        self.tipIds = [4, 8, 12, 16, 20]
        self.results = None # Aggiunto per evitare errori

    def findFingers(self, frame, draw=True):
        imgRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        self.results = self.hands.process(imgRGB)

        if self.results.multi_hand_landmarks:
            for handLms in self.results.multi_hand_landmarks:
                if draw:
                    self.mpDraw.draw_landmarks(frame, handLms, self.handsMp.HAND_CONNECTIONS)

        return frame

    def findPosition(self, frame, handNo=0, draw=True):
        xList, yList = [], []
        bbox = []
        self.lmsList = []

        if self.results and self.results.multi_hand_landmarks:
            if handNo < len(self.results.multi_hand_landmarks):
                myHand = self.results.multi_hand_landmarks[handNo]
                for id, lm in enumerate(myHand.landmark):
                    h, w, c = frame.shape
                    cx, cy = int(lm.x * w), int(lm.y * h)
                    xList.append(cx)
                    yList.append(cy)
                    self.lmsList.append([id, cx, cy])
                    if draw:
                        cv2.circle(frame, (cx, cy), 5, (255, 0, 255), cv2.FILLED)

                xmin, xmax = min(xList), max(xList)
                ymin, ymax = min(yList), max(yList)
                bbox = xmin, ymin, xmax, ymax

                if draw:
                    cv2.rectangle(frame, (xmin, ymin), (xmax, ymax), (255, 0, 255), 2)
```

```

        cv2.rectangle(frame, (xmin - 20, ymin - 20), (xmax + 20, ymax + 20), (0, 255, 0))

    return self.lmsList, bbox

def findFingerUp(self):
    fingers = []

    if not self.lmsList: # Evita errori se la lista è vuota
        return fingers

    # Pollice (controllo orizzontale)
    if self.lmsList[self.tipIds[0]][1] > self.lmsList[self.tipIds[0] - 1][1]:
        fingers.append(1)
    else:
        fingers.append(0)

    # Altre dita (controllo verticale)
    for id in range(1, 5):
        if self.lmsList[self.tipIds[id]][2] < self.lmsList[self.tipIds[id] - 2][2]:
            fingers.append(1)
        else:
            fingers.append(0)

    return fingers

def findDistance(self, p1, p2, frame, draw=True, r=15, t=3):
    if not self.lmsList: # Evita errori se la lista è vuota
        return None, frame, []

    x1, y1 = self.lmsList[p1][1:]
    x2, y2 = self.lmsList[p2][1:]
    cx, cy = (x1 + x2) // 2, (y1 + y2) // 2

    if draw:
        cv2.line(frame, (x1, y1), (x2, y2), (255, 0, 255), t)
        cv2.circle(frame, (x1, y1), r, (255, 0, 255), cv2.FILLED)
        cv2.circle(frame, (x2, y2), r, (255, 0, 0), cv2.FILLED)
        cv2.circle(frame, (cx, cy), r, (0, 255, 0), cv2.FILLED)

    distance = math.hypot(x2 - x1, y2 - y1)
    return distance, frame, [x1, y1, x2, y2, cx, cy]

def main():
    ptime = time.time()
    cap = cv2.VideoCapture(0)
    detector = HandTrackingDynamic()
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

    if not cap.isOpened():
        print("Cannot open camera")
        return

    while True:
        ret, frame = cap.read()
        if not ret:
            print("Failed to capture image")
            break

        frame = detector.findFingers(frame)
        lmsList, _ = detector.findPosition(frame)

        if len(lmsList) != 0:
            ctime = time.time()
            fps = 1 / (ctime - ptime)
            ptime = ctime
            cv2.putText(frame, str(int(fps)), (10, 70), cv2.FONT_HERSHEY_PLAIN, 3, (255, 0, 255), 3)

        cv2.imshow('Hand Tracking', frame)
        if cv2.waitKey(1) & 0xFF == ord('q'): # Premi 'q' per uscire
            break

    cap.release()
    cv2.destroyAllWindows()

```

```
if __name__ == "__main__":
    main()
```

Cosa è stato migliorato

- ✓ **Bug fix:** Evitati errori di accesso a liste vuote (`self.lmsList`).
- ✓ **Variabili rinominate:** Evitato l'uso di `len` come variabile.
- ✓ **Fps migliorato:** `ptime` inizializzato correttamente.
- ✓ **Exit pulito:** Il ciclo si chiude con il tasto `q`.
- ✓ **Controlli più sicuri:** `findPosition()` verifica che `handNo` non sia fuori range.

Ora il codice è più stabile e pronto per l'uso! 🚀



mi spieghi passo passo cosa questo codice

Certo! Ti spiegherò il codice **passo dopo passo**, suddividendolo in sezioni.

1. Importazione delle librerie

```
python

import cv2
import mediapipe as mp
import time
import math
```

- **cv2 (OpenCV):** Libreria per elaborazione di immagini e video.
- **mediapipe:** Libreria di Google per il tracciamento delle mani.
- **time:** Per calcolare gli FPS.
- **math:** Per calcoli matematici, come la distanza tra due punti.

2. Definizione della classe `HandTrackingDynamic`

Questa classe gestisce il tracciamento delle mani e il rilevamento delle dita.

```
python

class HandTrackingDynamic:
    def __init__(self, mode=False, maxHands=2, detectionCon=0.5, trackCon=0.5):
```

- **`mode=False`:** Se `True`, il modello elabora le immagini in modo statico, altrimenti lavora su video in tempo reale.

- ``maxHands=2``: Numero massimo di mani da rilevare.
- ``detectionCon=0.5``: Soglia di confidenza per il rilevamento della mano.
- ``trackCon=0.5``: Soglia di confidenza per il tracciamento della mano.

2.1 Inizializzazione di MediaPipe

```
python

`self.handsMp = mp.solutions.hands
self.hands = self.handsMp.Hands(static_image_mode=self.mode,
                                max_num_hands=self.maxHands,
                                min_detection_confidence=self.detectionCon,
                                min_tracking_confidence=self.trackCon)

self.mpDraw = mp.solutions.drawing_utils
self.tipIds = [4, 8, 12, 16, 20]
self.results = None # Evita errori se non ci sono mani rilevate`
```

- ``self.handsMp.Hands(...)``: Inizializza il modello per il tracciamento delle mani.
- ``self.mpDraw``: Permette di disegnare i punti delle mani.
- ``self.tipIds``: Contiene gli **ID** delle punte delle dita (pollice, indice, medio, anulare, mignolo).
- ``self.results = None``: Evita errori se la mano non viene rilevata.

3. Metodo ``findFingers()`` – Rileva la mano

```
python

`def findFingers(self, frame, draw=True):
    imgRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    self.results = self.hands.process(imgRGB)
`
```

- **Converte l'immagine in RGB** (MediaPipe lavora con RGB, OpenCV usa BGR).
- **Elabora l'immagine** e trova le mani con ``self.hands.process(imgRGB)``.

Se vengono rilevate mani:

```
python

`if self.results.multi_hand_landmarks:
    for handLms in self.results.multi_hand_landmarks:
        if draw:
            self.mpDraw.draw_landmarks(frame, handLms, self.handsMp.HAND_CONNECTIONS)
`
```

- ``multi_hand_landmarks`` contiene i punti della mano.
- **Disegna i punti e le connessioni** tra le dita.

👉 Restituisce l'immagine con la mano disegnata.

4. Metodo `findPosition()` – Trova le coordinate della mano

python

```
`def findPosition(self, frame, handNo=0, draw=True):
    xList, yList = [], []
    bbox = []
    self.lmsList = []
`
```

- Inizializza tre liste:
 - `xList, yList`: Per memorizzare le coordinate (x, y) di ogni punto della mano.
 - `bbox`: Per calcolare il rettangolo che circonda la mano.
 - `self.lmsList`: Memorizza tutti i punti chiave della mano.

Se MediaPipe rileva mani:

python

```
`if self.results and self.results.multi_hand_landmarks:
    if handNo < len(self.results.multi_hand_landmarks):
        myHand = self.results.multi_hand_landmarks[handNo]
`
```

- Controlla che ci siano mani rilevate.
- Se `handNo=0`, prende la prima mano rilevata.

Ora, estrae i punti della mano:

python

```
`for id, lm in enumerate(myHand.landmark):
    h, w, c = frame.shape
    cx, cy = int(lm.x * w), int(lm.y * h)
    xList.append(cx)
    yList.append(cy)
    self.lmsList.append([id, cx, cy])
    if draw:
        cv2.circle(frame, (cx, cy), 5, (255, 0, 255), cv2.FILLED)
`
```

- Converte le coordinate della mano da normalizzate (0-1) a pixel (`cx, cy`).
- Aggiunge i punti alla lista `self.lmsList`.
- Disegna un **cerchio** su ogni punto della mano.

Calcola il rettangolo che circonda la mano

python

```
`xmin, xmax = min(xList), max(xList)
ymin, ymax = min(yList), max(yList)
bbox = xmin, ymin, xmax, ymax
`
```


- Trova il **rettangolo più piccolo** che racchiude tutti i punti della mano.

Se ``draw=True``, disegna il rettangolo:

```
python

cv2.rectangle(frame, (xmin - 20, ymin - 20), (xmax + 20, ymax + 20), (0, 255, 0), 2)
```

👉 Restituisce ``self.lmsList`` (punti della mano) e ``bbox`` (rettangolo).

5. Metodo ``findFingerUp()`` – Riconosce quali dita sono alzate

```
python

def findFingerUp(self):
    fingers = []
    if not self.lmsList:
        return fingers
    `
```

- Controlla se ci sono dati nella lista ``self.lmsList``.

Pollice (controllo orizzontale):

```
python

if self.lmsList[self.tipIds[0]][1] > self.lmsList[self.tipIds[0] - 1][1]:
    fingers.append(1) # Pollice alzato
else:
    fingers.append(0) # Pollice abbassato
`
```

- Controlla se la punta del pollice (``self.tipIds[0]``) è a **destra** rispetto al punto di riferimento.

Altre dita (controllo verticale):

```
python

for id in range(1, 5):
    if self.lmsList[self.tipIds[id]][2] < self.lmsList[self.tipIds[id] - 2][2]:
        fingers.append(1) # Dito alzato
    else:
        fingers.append(0) # Dito abbassato
`
```

- Controlla se la punta del dito è **sopra** il secondo giunto della stessa mano.

👉 Restituisce una lista di 5 elementi con ``1`` se il dito è alzato, ``0`` se è abbassato.

6. Metodo ``findDistance()`` – Calcola la distanza tra due punti

python

```
`def findDistance(self, p1, p2, frame, draw=True, r=15, t=3):`
```

- Trova la distanza tra due punti `p1` e `p2`.

python

```
`x1, y1 = self.lmsList[p1][1:]`  
`x2, y2 = self.lmsList[p2][1:]`  
`cx, cy = (x1 + x2) // 2, (y1 + y2) // 2`
```

- Prende le coordinate `(x1, y1)` e `(x2, y2)`.
- Trova il punto **centrale** tra i due.

Se `draw=True`, disegna una **linea** e dei **cerchi** sui punti:

python

```
`cv2.line(frame, (x1, y1), (x2, y2), (255, 0, 255), t)`  
`cv2.circle(frame, (x1, y1), r, (255, 0, 255), cv2.FILLED)`  
`cv2.circle(frame, (cx, cy), r, (0, 255, 0), cv2.FILLED)`
```

👉 Restituisce la distanza tra i due punti.

7. Funzione `main()` – Avvia la webcam

python

```
`while True:`  
    ret, frame = cap.read()  
    frame = detector.findFingers(frame)  
    lmsList, _ = detector.findPosition(frame)
```

- **Cattura un frame** dalla webcam.
- **Rileva la mano** e i punti.

Se la mano è rilevata, calcola gli FPS:

python

```
`ctime = time.time()`  
`fps = 1 / (ctime - ptime)`  
`ptime = ctime`
```

Se premi 'q', esce dal programma:

```
python
```

```
`if cv2.waitKey(1) & 0xFF == ord('q'):  
    break  
`
```

 Il codice rileva le mani in tempo reale e mostra gli FPS sullo schermo.